

Phase 8 — Predictive Layer

LineTwin · Accenture Innovation Challenge 2026 · Round 2 · Problem Track 4 "DigitalTwin.ai"

Purpose

Build the two-model predictive layer: Model A (an offline benchmark on real public data, proving the modelling capability is real) and Model B (the live station-risk scorer, monotone by construction, running on the twin), plus rolling-horizon bottleneck prediction — genuinely underexplored territory, since only ~2 prior works do DT-based bottleneck *prediction* rather than detection.

This phase produced the label-generating process (`docs/DATA.md`) **before** any metric, per this project's standing rule. It also found five things worth recording in full: two citation corrections, one calibration bug that required redoing the bias solve, one genuine bug in the rolling-horizon fork, and one *suspected* bug that turned out not to be real — corrected rather than left as a false claim.

Two citation corrections, found while building this, not before

AI4I's failure-mode columns. `docs/CITATIONS.md` asserted they sum to 433. Verified directly via `fetch_ucirepo(id=601)` : **TWF 46 + HDF 115 + PWF 95 + OSF 98 + RNF 19 = 373**, not 433. Corrected in the ledger with the arithmetic shown, rather than carried forward unverified.

scikit-learn's MCC warning. The ledger claimed scikit-learn returns 0.0 *with a warning* for an undefined (0/0) MCC on a constant classifier. `ml/benchmark_public.py` catches warnings around exactly this call and asserts on what it actually sees — **no warning was raised** in the pinned scikit-learn version. Corrected to report what was observed, not what was assumed.

A bug found via this module's own tests, requiring the whole calibration to be redone

Writing `tests/test_features.py` 's bounds check (`blocked_fraction / starved_fraction` $\in [0, 1]$) immediately failed: **starved_fraction = 4.59** for a station that had been continuously STARVED for ~160s. Root cause: the first version of `FeatureExtractor` diffed `Station.time_in_state` 's cumulative totals between samples to approximate "time spent in state X this interval" — but `time_in_state` is only updated **at a state transition** (`sim/station.py` 's `_set_state`). A station holding one state for many consecutive samples shows a delta of zero each time, then dumps its *entire* accumulated duration into a single sample the instant it finally transitions.

Fixed by sampling `Station.state` directly at each tick and taking the fraction of samples in the window that were BLOCKED/STARVED — a presence estimator immune to how long any one state lasts, needing no knowledge of transition timing at all.

Consequence: `tools/calibrate_label_bias.py` had already been run once against the buggy extractor (`BIAS = -8.4183`). Re-run after the fix (`BIAS = -8.3743`, converged in the same 3 iterations) — a small shift, since the *mean* fraction over a long run turns out similar either way; it was the buggy version's *tail* behavior that was wrong, which the bug's own unit test target actually catches.

A suspected second bug in rolling-horizon prediction — corrected after re-checking, not left as a false claim

Building `fork_and_predict()`, an 8× perturbation applied to a normally-quiet station appeared to never propagate into the forecast: `units_completed` stayed frozen regardless of horizon. The prime suspect looked solid: queue depth was seeded by mutating `Store.items` directly rather than through `Store.put()`, and simpy's event machinery is normally put-triggered — a plausible mechanism for a station's already-pending `get()` never resolving.

Checked against simpy's own source before accepting the theory: `Store._do_get` just tests `if self.items:` synchronously, with no requirement that items arrived via a `put()` event. Re-tested at a longer horizon (900s instead of 300s): the station *had* progressed — the freeze was simply "not enough forecast time for one cycle at 8× the duration to complete," which is correct behavior, not a bug. The suspected fix was reverted from being claimed as one; both the code comment and the test file were corrected to record what was actually true rather than what first looked true. Recorded here so a future reader encountering the same symptom does not re-diagnose the same false lead.

The one genuine bug in that module: placeholder queue-seeding units used negative `unit_id`s. The instant one completes a cycle during the forecast, it flows through `Line.make_on_departure` like any real unit and builds a `UnitEvent` — whose `unit_id` field requires `ge=0` (`contracts.py`). A pydantic `ValidationError` inside a simpy process surfaced as a confusing secondary `TypeError` from simpy's own exception re-raising. Fixed with a large, non-negative placeholder offset.

A real, non-obvious finding from rolling-horizon prediction: momentary inertia

Applying an 8× perturbation to a normally-quiet station and forecasting at increasing horizons:

Horizon	Predicted bottleneck
300s	S13 (unrelated to the perturbation)
900s	S13
2000s	S13

Horizon	Predicted bottleneck
4000s	S22 (the perturbed station)
8000s	S22

This is correct behavior, not a bug: the momentary Active Period Method names whichever *currently active* station's active period started earliest. S13 already had a long-running active period at the moment the perturbation was applied (a known live contender, Phases 5–7); S22's newly-elevated cycle time only accumulates its own active period from that moment forward. The forecast correctly shows S13 retaining the "bottleneck" label for a while — inertia from an existing head start — until enough forecast horizon passes for S22's own active period to overtake it. Documented and asserted directly in `tests/test_rolling_horizon.py`, at both a horizon too short (S13 persists) and long enough (S22 wins).

Model A: the offline benchmark (AI4I 2020, real data)

Model	PR-AUC	ROC-AUC	Brier	MCC@t	note
always_zero	0.0340	0.5000	0.0340	0.0000	MCC undefined (0/0); no warning raised (verified, see above)
logistic_floor	0.3798	0.8885	0.1190	0.4058 @ 0.83	
xgboost_uncalibrated (scale_pos_weight=28.5)	0.7726	0.9722	0.0148	0.7502 @ 0.59	
xgboost_calibrated_isotonic	0.7949	0.9696	0.0130	0.7686 @ 0.38	best real model
smote_then_cv (inflated)	0.9988	n/a	n/a	n/a	WRONG methodology, published on purpose
smote_then_real_holdout	0.7455	0.9685	0.0190	0.6870 @ 0.67	same model, real test set

The deliberate SMOTE failure case: applying SMOTE before cross-validating inflates PR-AUC to 0.9988 — a **25.4% overstatement** versus the same model's 0.7455 on the real, untouched held-out set. Published as the headline result of this comparison, not a footnote: the collapse between the two numbers *is* the demonstration.

Accuracy appears in this codebase exactly once — in the sentence explaining why it is never reported (3.39% prevalence means all-zero scores >96% accuracy while catching zero failures).

Model B: the live scorer

Trained on configs A, B, C (108,678 rows); calibrated and threshold-tuned on config D (37,320 rows); evaluated on config E, **held out and untouched until this step** (33,573 rows).

	PR-AUC on config E (UNSEEN)
Model B (monotone XGBoost + isotonic)	0.0264
Single-feature <code>cycle_time_z</code> logistic baseline	0.0182
Lift	+44.7%

Both numbers are modest in absolute terms (defect prevalence on config E is 0.5%), but the honest-lift gate is what matters here: **44.7% lift clears the 10% bar** with real margin, so this is reported as a genuine, if small-absolute-magnitude, combiner — not reframed as marginal.

Monotonicity, verified empirically across all five features (`tests/test_scorer.py`), not merely imposed by the training flag: sweeping each feature from its typical range while holding the others at baseline, calibrated risk is non-decreasing everywhere, for every one of the five features independently.

Deliverables produced

Artefact	What it does
<code>docs/DATA.md</code>	The label-generating process, written before any metric
<code>src/twin/risk/features.py</code>	<code>FeatureExtractor</code> — shared, non-invasive rolling-window sampler (fixed presence-sampling bug documented above)
<code>src/twin/risk/labels.py</code>	<code>oracle_risk()</code> , the synthetic label function, calibrated bias
<code>tools/calibrate_label_bias.py</code>	Fixed-point solver for the one calibrated constant
<code>tools/generate_training_data.py</code>	179,571 rows across 5 configs, disjoint by construction
<code>ml/benchmark_public.py</code>	Model A on AI4I 2020, including the deliberate SMOTE failure case
<code>tools/train_station_risk.py</code>	Model B training: monotone XGBoost, isotonic calibration, honest-lift gate
<code>src/twin/risk/scorer.py</code>	Live inference: <code>StationRiskScorer</code> , exact TreeSHAP top-2 drivers, always tagged associative
<code>src/twin/diagnostic/rolling_horizon.py</code>	Fork-and-forecast rolling-horizon bottleneck prediction
<code>src/twin/sim/engine.py</code> (extended)	Risk scoring wired at 1Hz regardless of tick rate, called directly (not <code>to_thread</code> — a microsecond-scale predict, the opposite case from Phase 7's Tukey-Kramer fix)

Artefact	What it does
6 new test files	26 new tests: labels, features, benchmark helpers, import hygiene, no-config-leak, scorer monotonicity, rolling-horizon

One explicit limitation carried forward, not solved here

Model B's live features are computed from full internal simulation state for **all 30 stations**, including the 8 uninstrumented ones — Phase 9's sensor-gap layer does not exist yet and has not been wired into this feature pipeline. A real deployment's dark-station risk score would need to be computed from *inferred* features (Phase 9's harmonic extension), not ground truth. Stated here as an interim state, to be revisited explicitly when Phase 9 builds the inference layer.

Exit criteria

Criterion	Status
Label-generating process documented before any metric	Met — docs/DATA.md
Benchmark on real public data, including a deliberate failure case	Met — SMOTE collapse: 25.4% overstatement
Live model: monotone constraints, calibration, held-out config split	Met — config E never touched until final evaluation
Honest-lift gate: baseline published beside the model	Met — +44.7% lift, clears the 10% bar
MCC always with threshold; accuracy reported exactly once, explained	Met
Rolling-horizon bottleneck prediction	Met — fork/forecast mechanism, momentary-inertia finding documented and tested
No config leakage; monotonicity holds	Met — test_no_config_leak.py , test_risk_is_monotone_non_decreasing_in_each_feature (5/5 features)

134/134 tests passing (108 from Phases 1–7 + 26 new this phase).

Next

Phase 9 — Sensor Gaps, Placement & Genealogy. Harmonic extension across the line graph for the 8 uninstrumented stations; the exact partition-of-unity evidence attribution; defect genealogy with transfer-delay realignment. Directly closes this phase's one carried-forward limitation: once Phase 9's inferred features exist, Model B's feature pipeline should be revisited to use them for dark stations rather than ground truth.